

Практика 1. Программирование в λ -исчислении

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- + Синтаксис λ -исчисления
- + Свободные и связанные переменные
- + Классические комбинаторы
- + Бета-редукция
- + α , β , η -эквивалентности

Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☁)

Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☀)

Отчётность

- ▶ Оценка за курс — оценка за экзамен (успеваемость по практике тоже учитывается)
- ▶ Допуск на экзамен — зачёт по всем домашкам и летучкам, кроме, может быть, двух
- ▶ Правила зачёта по домашкам расписаны в самих домашках
- ▶ Отчётность будет вестись в гугл-табличке, правила пользования:
 - ▶ Не сравниваем свою успеваемость с другими и не переживаем
 - ▶ Смотрим, к кому можно обратиться за помощью
 - ▶ Перфекционизм — зло

Домашки

- ▶ Выдаются на каждую неделю
- ▶ В конце семестра будет достаточно возможностей добрать баллов
- ▶ За частичные решения тоже даются баллы
- ▶ После пары старайтесь как можно скорее начать делать задания
- ▶ В начале каждой пары будет разбор

NB Не столь важно не конечное решение как процесс поиска

Коммуникация online

- ▶ Личка преподавателей в телеграме
- ▶ Общий чат в телеграме¹

¹Все ссылки на wiki.

Коммуникация offline

- ▶ Лучше спросить сразу: “потом” не будет времени, и сил не прибавится
- ▶ Спрашивать — ответственность и работа студентов
- ▶ Распределять время и оценивать уместность — ответственность и работа преподавателя
- ▶ Если совсем трудно, пишите в чат/личку во время пары или подходите после пары
- ▶ На парах “присутствуйте”, а не занимайтесь отвлечёнными делами

Условные обозначения

- (💡) — дополнительная общеобразовательная информация, данная для контекста
- (*) — продвинутый материал, не обязательный для понимания в рамках курса
- (⚙️) — заметки, полезные для практического применения

Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☀)

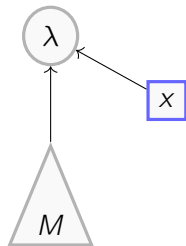
Синтаксис λ-исчисления

Множество программ (термов) — Λ , множество допустимых переменных — V .
Синтаксис задаётся индуктивным определением:

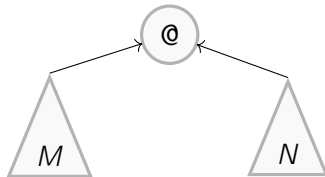
Переменные $x \in \Lambda$, если $x \in V$



Абстракция $(\lambda x. M) \in \Lambda$, если $x \in V, M \in \Lambda$



Аппликация $(M N) \in \Lambda$, если $M \in \Lambda, N \in \Lambda$



Опускаем скобочки

$$\lambda f\ g.\ \lambda x.\ f\ (g\ x)$$

todo Какие скобочки пропущены в этой записи?

todo Можно ли опустить имеющиеся?

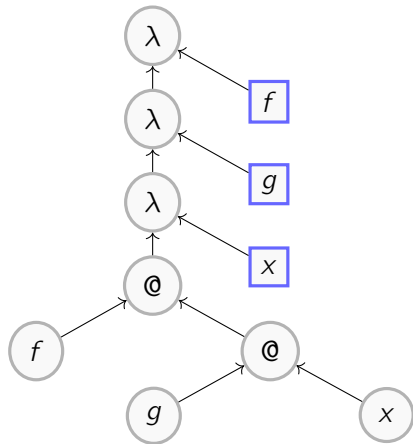
todo Как выглядит дерево синтаксиса, соответствующее этому терму?

todo Опустите скобочки $((((a)\ b)\ (c\ d))\ (e))$

todo Опустите скобочки $(y)\ (\lambda x.\ (x))$

todo Опустите скобочки $(\lambda x.\ (\lambda y.\ (x\ y)))\ (a)\ (b)$

todo Опустите скобочки $\lambda x.\ (x\ (\lambda y.\ y)\ z)$



Свободные и связанные переменные

$$\lambda x. (\lambda x y. x) (x y)$$

todo Какие переменные свободные в этом терме?

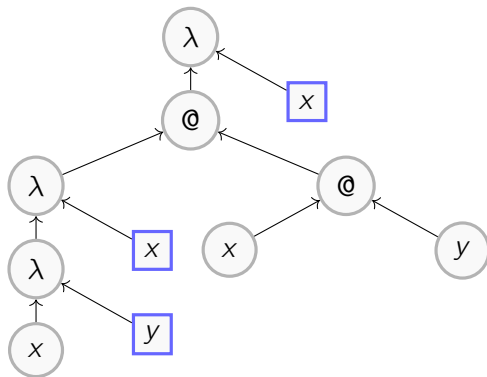
todo Какими связывателями связаны остальные?

todo А для $\lambda x. x$ ($\lambda z. x z$) z ?

todo А для $\lambda a. \lambda b. \lambda c. a b (b c)$ c ?

todo А для $a (a b) \lambda a. a a$?

todo А для $\lambda a. (\lambda b. a) b$?



Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☁)

Подстановка

Определение

$$\left\{ \begin{array}{l} [x \mapsto N] x \equiv N \\ [x \mapsto N] y \equiv y \\ [x \mapsto N] P Q \equiv ([x \mapsto N]P) ([x \mapsto N]Q) \\ [x \mapsto N] \lambda x. P \equiv \lambda x. P \\ [x \mapsto N] \lambda y. P \equiv \lambda y. [x \mapsto N] P, \quad y \notin FV(N) \\ [x \mapsto N] \lambda y. P \equiv \lambda y'. [x \mapsto N] ([y \mapsto y']P), \quad y \in FV(N) \end{array} \right.$$

todo $[x \mapsto y] x (\lambda y. y z)$

 $y (\lambda y. y z)$

todo $[x \mapsto y z] x (\lambda y. y z)$

 $y z (\lambda y. y z)$

todo $[y \mapsto x] x (\lambda y. y z)$

 $x (\lambda y. y z)$

todo $[z \mapsto x] x (\lambda y. y z)$

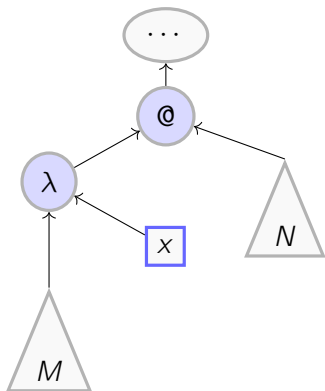
 $x (\lambda y. y x)$

todo $[z \mapsto x y] x (\lambda y. y z)$

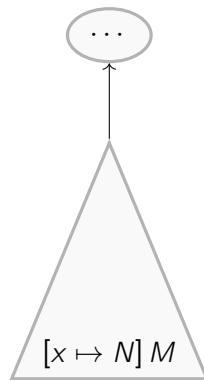
 $x (\lambda y'. y' (x y))$

β-редукция — применение функций

Находим редекс:



Сокращаем редекс:



todo $I (K I) I$

todo $B (\lambda x. x) (\lambda x. x x) a$

todo $B S K$

$\rightsquigarrow (\lambda x. x) (K I) I \rightsquigarrow (K I) I \rightsquigarrow ((\lambda x y. x) I) I \rightsquigarrow (\lambda y. I) I \rightsquigarrow I$

$\rightsquigarrow (\lambda x. x) ((\lambda x. x x) a) \rightsquigarrow (\lambda x. x) (a a) \rightsquigarrow a a$

$\rightsquigarrow \lambda x. S (K x) \rightsquigarrow \lambda x. S (\lambda y. x) \rightsquigarrow \lambda x g z. (\lambda y. x) z (g z) \rightsquigarrow B$

Эквивалентности

Эквивалентны ли следующие термы?

todo $x =_? x$

 синтаксическое равенство

todo $x =_? y$

 нет

todo $\lambda x. x =_? \lambda y. y$

 α -эквивалентность

todo $\lambda x. x =_? (\lambda y. y) (\lambda x. x)$

 β -эквивалентность

todo $\lambda x y. x y =_? \lambda x. x$

 η -эквивалентность

Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☀)

Значения-записи (records, tuples)

Одна сущность может описываться несколькими различными свойствами:

- ▶ Комментарий в соцсети: автор, пост, текст, количество лайков...
- ▶ Котик: имя, мурлычность (очевидно)

Пример: Python

```
class Cat:
    def __init__(self, name, murlicity):
        self.name = name
        self.murlicity = murlicity
```

Пример: C++

```
struct Cat {
    string name;
    int murlicity;
};
```

Что мы хотим от структуры данных?

Конструкторы

Создают новые значения структуры данных:

```
barsik = Cat(name = "barsik", murlicity = 4)
```

Элиминаторы

Получают (используют) информацию из структуры данных:

```
barsik.murlicity
```

Мутаторы

Изменение информации в экземпляре структуры данных:

```
barsik.murlicity = barsik.murlicity + 3
```

В ФП не принято пользоваться изменяемыми данными, и мутаторы не нужны¹:

```
happyBarsik = Cat(name = barsik.name,  
                  murlicity = barsik.murlicity + 3)
```

¹[Roman Elizarov: Immutability we can afford.](#)

Моделируем котиков

Фактически нужно научиться задавать пары (имя + мурлычность):

API пар

- ▶ Терм-конструктор **pair**
- ▶ Элиминаторы **fst** и **snd**

Законы пар

1. $\forall N M. \mathbf{fst} (\mathbf{pair} N M) \equiv_{\beta} N$
2. $\forall N M. \mathbf{snd} (\mathbf{pair} N M) \equiv_{\beta} M$

Кодируем котиков

Эм. Как??



Пары в λ -исчислении

Законы пар

1. $\forall N M. \mathbf{fst} (\mathbf{pair} N M) \equiv_{\beta} N$
2. $\forall N M. \mathbf{snd} (\mathbf{pair} N M) \equiv_{\beta} M$

Конструктор

- ▶ Принимает аргументы
- ▶ Принимает функцию действующую на всей информации пары
- ▶ $\mathbf{pair} := \lambda x y. \lambda f. f \ x \ y$
- ▶ Благодаря каррированности, термы вида $\lambda f. f \ n \ m$ можно считать парами

Элиминаторы

- ▶ $\mathbf{fst} := \lambda p. p \ \mathbf{K}$
- ▶ $\mathbf{snd} := \lambda p. p \ \mathbf{K}^*$

Проверка законов пар

todo Проверьте, что реализация пар удовлетворяет законам

todo Определите конструктор и элиминаторы для тройки двумя способами, каким законам они должны удовлетворять?



$$\begin{aligned}
 \text{fst } (\text{pair } \boxed{N} M) &\rightsquigarrow_{\beta} N \\
 \text{fst } ((\lambda x y. \lambda f. f \ x \ y) \ N \ M) &\rightsquigarrow_{\beta} N \\
 \text{fst } (\lambda f. f \ N \ M) &\rightsquigarrow_{\beta} N \\
 (\lambda p. p \ \mathbf{K}) (\lambda f. f \ N \ M) &\rightsquigarrow_{\beta} N \\
 (\lambda f. f \ N \ M) \ \mathbf{K} &\rightsquigarrow_{\beta} N \\
 \mathbf{K} \ N \ M &\rightsquigarrow_{\beta} \boxed{N}
 \end{aligned}$$

► Теперь можно пользоваться правилом большого шага:

$$\text{fst } (\text{pair } N \ M) \rightarrow N$$



- Пара пар: **triple** = $\lambda x \ y \ z. \text{pair } x \ (\text{pair } y \ z)$; **snd3** = $\lambda p. \text{fst } (\text{snd } p)$
- Тройка тройка: **triple** = $\lambda x \ y \ z. \lambda f. f \ x \ y \ z$; **snd3** = $\lambda p. p \ (\lambda x \ y \ z. y)$

Каррирование и раскаррирование

todo Как выразить функцию многих аргументов?



Пусть принимает кортеж аргументов!

todo Реализуйте “переключение” между стилями



curry $:= \lambda f. \lambda a \ b. f \ (\mathbf{pair} \ a \ b)$

uncurry $:= \lambda f. \lambda p. f \ (\mathbf{fst} \ p) \ (\mathbf{snd} \ p)$

Me : I really want to eat curry

**University: We have Curry at
University**

Curry at the University:

$X \times Y \rightarrow Z = X \rightarrow (Y \rightarrow Z)$

Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☀)

Моделируем логические величины

API логических величин

- ▶ Конструкторы: две константы **true** и **false**
- ▶ Элиминатор: **if**

Законы для логической величины

1. **if true** $T \ E =_{\beta} T$
2. **if false** $T \ E =_{\beta} E$

Логические величины в λ -исчислении

Законы логических величин

1. **if true** $T \ E =_{\beta} T$
2. **if false** $T \ E =_{\beta} E$

Конструкторы

- ▶ Принимают ноль аргументов (константы)
- ▶ Принимают два потенциальных результата, из которых выбирают нужный
- ▶ **true** $:= \lambda t \ e. t$ (или **K**)
- ▶ **false** $:= \lambda t \ e. e$ (или **K***)

Элиминатор

- ▶ Вся логика уже в значениях, осталось применить
- ▶ **if** $:= \lambda b \ t \ e. b \ t \ e$
- ▶ Или даже **if** $:= \lambda x. x$ (или **if** $:= \text{id}$)

Проверка законов логических величин

todo Проверьте выполнение законов



$$\begin{aligned}
 & \text{if true } \boxed{\top} E \rightsquigarrow_{\beta} \\
 & (\lambda b t e. b t e) \text{ true } T E \rightsquigarrow_{\beta} \\
 & \text{true } T E \rightsquigarrow_{\beta} \\
 & \mathbf{K} T E \rightsquigarrow_{\beta} \\
 & (\lambda x y. x) T E \rightsquigarrow_{\beta} \boxed{\top}
 \end{aligned}$$

Программирование операций над логическими величинами: отрицание

b	$\neg b$
true	false
false	true

На Python

```
def notb(b):  
    if b:  
        return False  
    else:  
        return True
```

На λ -исчислении**not** $b := \text{if } b \text{ false true}$ (подразумеваем **not** $:= \lambda b. \text{if } b \text{ false true}$)

Программирование операций над логическими величинами: конъюнкция

x	y	$x \wedge y$
true	true	true
true	false	false
false	true	false
false	false	false

На Python

```
def andb(x, y):  
    if x:  
        return y  
    else:  
        return False
```

На λ -исчислении

and $x\ y := \text{if } x\ y\ \text{false}$

Или эквивалентный вариант:

and $x\ y := \lambda t\ e.\ x\ (y\ t\ e)\ e$

Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☀)

Натуральные числа $\mathbb{N} := \{0, 1, 2, 3, \dots\}$

API логических величин

Первая аксиома Пеано: ноль является натуральным числом (**0**)

Вторая аксиома Пеано: число, следующее за натуральным, тоже натуральное (**suc** n)

Элиминатор Воспользоваться натуральным числом — что-то сделать n раз: **natElim**

Законы для натуральных чисел

1. $\forall f\ ini. \mathbf{natElim\ 0\ } f\ ini =_{\beta} ini$
2. $\forall n\ f\ ini. \mathbf{natElim\ (suc\ } n) f\ ini =_{\beta} f\ (\mathbf{natElim\ } n\ f\ ini)$

Конструирование чисел Чёрча

Конструктор **0**

- ▶ Принимает ноль аргументов (константа)
- ▶ Принимает действие, начальный элемент и возвращает начальный элемент
- ▶ $\mathbf{0} := \lambda s \ z. z$

Конструктор **suc**

- ▶ Принимает один аргумент
- ▶ Принимает действие и начальный элемент, возвращает применение нужное число раз

$$\begin{aligned}
 \mathbf{suc} \ n &= \lambda s \ z. s \left(\underbrace{s \left(\cdots \left(s \left(z \right) \cdots \right) \right)}_{n+1} \right) \\
 &= \lambda s \ z. s \left(\underbrace{s \left(\cdots \left(s \left(z \right) \cdots \right) \right)}_n \right) = \lambda s \ z. s \ (n \ s \ z) \\
 \mathbf{suc} \ n &= \lambda s \ z. s \left(\underbrace{s \left(\cdots \left(s \left(z \right) \right) \cdots \right)}_{n+1} \right) \\
 &= \lambda s \ z. s \left(\underbrace{\cdots \left(s \left(s \ z \right) \cdots \right)}_n \right) = \lambda s \ z. n \ s \ (s \ z)
 \end{aligned}$$

Элиминация чисел Чёрча

Форма чисел Чёрча

Снова элиминация закодирована с самих значениях:

$$\begin{aligned}
 \mathbf{0} &:= \lambda s \ z. z \\
 \mathbf{1} &:= \lambda s \ z. s \ z \\
 \mathbf{2} &:= \lambda s \ z. s \ (s \ z) \\
 \vdots & \\
 \forall k, \mathbf{k} &:= \lambda s \ z. \underbrace{s \ (s \ (\dots s \ (z)))}_{k} \dots
 \end{aligned}$$

Элиминирование чисел Чёрча

Просто применить значение к аргументам:

$$\begin{aligned}
 \mathbf{natElim} \ n \ f \ ini &:= n \ f \ ini \\
 \text{или} \quad \mathbf{natElim} &:= \mathbf{id}
 \end{aligned}$$

Операции над числами Чёрча: сложение

todo Закодируйте сложение

⚡ Это всё равно что n раз увеличить m на единицу:

plus $n\ m := \text{natElim}\ n\ \text{suc}\ m$

⚡ Это всё равно что сначала отсчитать m , а потом от этого отсчитать n :

plus $n\ m := \lambda\ s\ z. n\ s\ (m\ s\ z)$

Операции над числами Чёрча: проверка чётности числа (1)

Методология

1. Смотрим, как терм ведёт себя в базовых случаях

isEven 0 $\rightsquigarrow_{\beta}$ **true**

isEven 1 $\rightsquigarrow_{\beta}$ **false**

isEven 2 $\rightsquigarrow_{\beta}$ **true**

...

- ▶ Функция принимает два значения
- ▶ Первый раз — **true**
- ▶ Далее значения чередуются

2. Вспоминаем, как воспользоваться числом:

natElim 3 $f\ ini \rightsquigarrow_{\beta} f\ (f\ (f\ ini))$

- ▶ Требуется сконструировать подходящие f и ini

Операции над числами Чёрча: проверка чётности числа (2)

Решение 1

- ▶ В качестве состояния будем поддерживать пару значений
- ▶ Начальное состояние: **start** := **pair true false**
- ▶ Переход: **swap** := $\lambda p. \text{pair } (\text{snd } p) (\text{fst } p)$
- ▶ Всё вместе: **isEven** $n := \text{fst } (\text{natElim } n \text{ swap start})$

Проверка:

- ▶ **isEven 0** $\rightsquigarrow_{\beta} \text{fst start} \rightsquigarrow_{\beta} \text{true}$
- ▶ **isEven 3** $\rightsquigarrow_{\beta} \text{fst } (\text{swap } (\text{swap } (\text{swap start}))) \rightsquigarrow_{\beta} \text{fst } (\text{swap } (\text{swap } (\text{pair false true}))) \rightsquigarrow_{\beta} \text{fst } (\text{pair false true}) \rightsquigarrow_{\beta} \text{false}$

Решение 2

- ▶ Состояние: булево значение
- ▶ Начальное значение: **true**; переход: **not**
- ▶ Всё вместе: **isEven** $n := \text{natElim } n \text{ not true}$

Содержание

1. Метаинформация о курсе
2. λ -исчисление
 - ▶ Синтаксис λ -исчисления
 - ▶ Семантика λ -исчисления
3. Кодирование данных в λ -исчислении
 - ▶ Значения-записи (records, tuples)
 - ▶ Логические величины
 - ▶ Натуральные числа
4. Материалы (☀)

Каррирование

Эквивалентен ли код?

$$\mathbf{K}^* := \lambda x \ y. y$$

Нет:

$$\mathbf{K}^* \ q \rightsquigarrow_{\beta} (\lambda x \ y. y) \ q \rightsquigarrow_{\beta} \lambda y. y$$

```
def k_star(x, y):  
    return y
```

```
> k_star(q)
```

```
TypeError: k_star() missing 1 required  
positional argument: 'y'
```

Правильная трактовка: “ $\mathbf{K}^*$ — функция, которая игнорирует свой аргумент и возвращает функцию, которая возвращает свой аргумент”:

$$\lambda x \ y. y \equiv \lambda x. \lambda y. y$$

```
def k_star(x):  
    def helper(y):  
        return y  
    return helper
```

Каррирование

Преобразование функции многих аргументов в набор вложенных функций, каждая из которых принимает часть аргументов исходной.¹

¹В ООП методы класса каррированные — часть их “аргументов” передаётся через конструктор.

Замыкания²

Что если закодировать не K^* , а K :

$$K := \lambda x. \lambda y. x$$

```
def k(x):  
    def helper(y):  
        return x  
    return helper
```

Замыкание (closure)

Пара из функции и окружения (состояния, к которому код функции имеет доступ).
Используется в прикладных языках как эффективное представление λ -функций.

Ещё пример на Python

```
screen = Screen(...)  
def handler():  
    screen.textField = ""  
clearButton.onClick(handler)
```

Ещё пример на C++

```
auto screen = Screen::create(...);  
clearButton.onClick([screen]() {  
    screen.textField = "";  
});
```

²<https://groups.seas.harvard.edu/courses/cs153/2018fa/lectures/Lec12-Functions.pdf>

Функции высших порядков

Функция высших порядков (high-order functions, HOF)

Принимает другие функции в качестве аргументов.

Пример на λ -исчислении

$$\mathbf{B} := \lambda[f]. \lambda[g]. \lambda x. f (g x)$$

Пример на Python

На практике не всегда каррирование по всем аргументам оправдано:

```
def compose(f, g):  
    def helper(x):  
        return f(g(x))  
    return helper
```

Пример на C++

```
function<int(int)> compose(  
    function<int(int)> f,  
    function<int(int)> g  
) {  
    return [f, g](int x) {  
        return f(g(x))  
    }  
}
```

Законы и algebra driven design⁴ (☁️)

Закодировав пару, получили абстракцию по данным:

- ▶ Не два значения, а доменная сущность
- ▶ Можем выбрать произвольную реализацию, удовлетворяющую законам
- ▶ Можем забыть, как устроена реализация и разгрузить голову
- ▶ Можем в произвольный момент подменить реализацию на другую (например, на более эффективную), и пользовательский код не сломается
- ▶ Можем автоматически тестировать реализации на соответствие законам³

³ John Hughes - Keynote: How to specify it! (...) - Lambda Days 2020

⁴ Sandy Maguire, "Algebra driven design"

“Кодирование наоборот”⁵ (☼)

- ▶ Пусть функции не возвращают результат, а принимают способ его сразу обработать
- ▶ Функции не продают, но показывают
- ▶ “Штуку я не дам, скажи, зачем она тебе, я всё сделаю”
- ▶ Вместо передачи объекта к месту деконструирования, передаём деконструирование к месту создания объекта
- ▶ Вместо $(X) \rightarrow R$ теперь $(X, (R) \rightarrow a) \rightarrow a$

⁵“Наоборот” также известно как tagless final, [church-boehm-berarducci encoding](#), CPS transformation.

Кодирование наоборот как оптимизация (☺)

- ▶ Это общий способ избавления от аллокаций и структур данных
- ▶ Пример с HFT⁶ доклада — оптимальный десериализатор:
 - ▶ Есть два вида сообщений
 - ▶ Десериализатор по-хорошему должен аллоцировать `std::variant`, но

С аллокацией

```
std::variant<Msg1, Msg2>
deserialize(bytes bs) {
    if (...) {
        return std::variant{Msg1(...)};
    } else {
        return std::variant{Msg2(...)};
    }
}
```

Без аллокации

```
template<class Impl>
auto deserialize(bytes bs) {
    if (...) {
        return Impl::processMsg1(...);
    } else {
        return Impl::processMsg2(...);
    }
}
```

⁶High frequency trading — очень быстро торгуют на бирже и зарабатывают деньги.

Что посмотреть в транспорте

- ▶ Функциональное программирование как предмет и метод теоретической информатики.
- ▶ Podlodka #279 — Тьюринг-полнота | Виталий Брагилевский.
- ▶ Blogpost: Roman Elizarov: Immutability we can afford.
- ▶ Памятка с полезностями от Computer Science Center

- ▶ Записи лекций Дениса Николаевича на МКН
- ▶ Книга: Ламбда-исчисление, Барендрегт
- ▶ Beyond Church encoding: Boehm-Berarducci isomorphism of algebraic data types and polymorphic lambda-terms
- ▶ Church-boehm-berarducci encoding